Assignment 1: Applied Cryptography – The Garage Door Protocol

Overview

Consider a garage door attached to a house. The garage door cannot be opened aside from a remote control in a vehicle, and the door from the garage to the house is thus often left unlocked. In the context of a "weakest link", opening the garage door may very well be the easiest way to enter a house.

A traditional way that a garage door opener works is to emit a secret that opens the door. This assignment consists of a number of different versions of the garage-door opening protocol, each of which is vulnerable to an active attacker. Your job is to observe successful protocols using the `garage_observer` tool and then perform an attack using `garage_attacker`.

You will perform eavesdropping, replay attacks, and cryptographic malleability (bit-flipping) attacks to break the security. A successful attack will have the garage door respond with:

```
SECURITY BREACH: DOOR OPEN! code: XXXXXX
```

Where `XXXXXX` is a hex string (e.g., `18A7874F`) that you submit to show you have attacked the door successfully.

Submission Instructions

For each version of the assignment, submit the following:

1. **The Code:** The hex code you received upon a successful breach.

2. **Vulnerability Analysis:** A paragraph explaining the specific cryptographic weakness in that version and how your attack exploited it.

The Basic Protocol

While different versions use different encryption or challenges, the underlying communication logic remains the same. The basic protocol is as follows:

1. Create a TCP connection to the garage server.

2. Issue a **connect** command to start the session.

3. Issue commands to the door. Once the session is established, you may issue any combination of the following commands in any order:

- `open door`
- `close door`
- `turn on lights`
- `turn off lights`

4. Issue a **disconnect** command to end the session.

5. Close the TCP connection.

Note: We have programmed both the observer and the attacker to automatically send a single identification character immediately after connecting ('o' for Observer and 'a' for Attacker). This is strictly for grading purposes. If you see an initial back-and-forth in plain text between the client and server, you can simply ignore it.

System Architecture: Servers and Observers

Each version of the garage door comes with two programs: a **server** and an **observer**.

- **Garage Server**: Simulates the door itself. It takes a command line argument representing the last four digits of your UCID (e.g., `./server_v1 5678`).
- **Garage Observer**: Simulates a legitimate user interacting with the door. It takes the IP address of the server and the last four digits of the UCID.

We have provided a Docker environment to ensure these programs run consistently across different student systems. Detailed setup instructions are provided later in this document.

How the Simulation Works

The observer connects to the IP and port of the garage server and issues a few commands based on the protocol. It outputs what is "observed" and the timestamp of that observation (e.g., "Lights turned on at 2026-01-14 11:57:01").

The observer program simulates someone else having a real-world interaction with the door while you act as the attacker observing the "airwaves." In the real world, this could be a neighbor capturing traffic from a car broadcasting connection commands to the garage door.

Capturing Traffic

We run the **server** and **observer** in different Docker containers to simulate them running on different machines. When running the `garage_observer`, you should record the network traffic using `tcpdump`. You can run this as a startup option to ensure you have a record of the network traffic. Note that the `.pcap` file recorded by the machine may include other communication from the container; you will need to filter out irrelevant traffic. You can use the specific port number used in the connection to isolate traffic between the observer and the server.

Analyzing Traffic in Wireshark

Once you have captured the network traffic using `tcpdump`, you will have a `.pcap` file. To analyze this data effectively, use **Wireshark**, which provides a visual interface for inspecting packet details. Launch Wireshark and open the `.pcap` file you generated.

In Wireshark, you can right-click a relevant packet, select **"Follow"**, and then **"TCP Stream"** to see the exact back-and-forth communication of TCP payload data.

A Note on Time

Wireshark may display timestamps relative to the start of the capture. To correlate network packets with the "observed events" output by the `garage_observer` (which uses absolute time), you should switch Wireshark to absolute time:

1. Right-click the "Time" column header.
2. Go to "Edit Column".
3. Change the display type to **Absolute Time**.

This allows you to match events (e.g., "Garage door opened") to contemporaneous network traffic. Note that there may be a small difference in time (less than a second) or a round number of hours due to timezone differences. You will get a sense of this clock skew in Version 1, where there is no encryption.

Filtering by Port

The capture file may contain background traffic that is irrelevant to the assignment. To see only the traffic between the observer and the server, locate the filter bar at the top of the Wireshark window and type:

```
tcp.port == <PORT>
```

(Replace `<PORT>` with the specific port number the garage server is using).

Attack Methodology

Once you have analyzed the traffic and figured out how to attack the system, you can connect directly to the IP and port to send your malicious traffic. Be sure to read the server's reply to capture your success code.

The goal for each version is to execute the following sequence:

1. Connect via TCP to the garage server.
2. Issue a connect command.
3. Issue an **open door** command.

4. Issue a disconnect command.

5. Close the TCP connection.

You are free to run the observer program as many times as you want. This may be necessary because its behavior (and the resulting traffic) can change across different runs.

The Garage Attacker Program

To assist you, we have provided a program called `garage_attacker`.

- **Usage:** It takes the IP and **port** (not the UCID) of the garage server as arguments. Note that each server version runs on a different port, which is displayed in the server output. Make sure to record this port number so you can use it later when launching the attacker.
- **Function:** It reads a line of **hex-encoded input** and sends the actual bytes over the socket. This is essential for later versions where data is encrypted.
- **Replaying Traffic:** You can select a data field in Wireshark, right-click, and choose **Copy > ...as a Hex Stream**. These bytes are formatted exactly how `garage_attacker` expects them. We recommend saving these hex streams in a text editor so you can easily reuse or edit them later.

Strategy

You may find it useful to restart your environment or delete old `.pcap` files between exercises to keep your analysis clean.

Before attacking, try to think about how you will break the protocol by reading the description. If an attack isn't obvious, look at the network traffic for clues. Remember key concepts from Unit 1: **replay attacks, cryptographic malleability, known plaintext, two-time pads, and information leakage**. Rule out concepts that clearly do not apply, and focus on the remaining possibilities.

Environment Setup and Running the Code

1. Prerequisites and Download the Files

This assignment requires the following tools:

1. **Docker:** To run the server and client environments. [Download Docker Desktop](https://www.docker.com/products/docker-desktop/).

* *Verification:* After installation, open a terminal and run `docker --version` to ensure it is installed and running.

2. **Wireshark:** To analyze the network traffic and extract hex codes. [Download Wireshark](https://www.wireshark.org/).

Please install both for your operating system before proceeding.

The binaries and resources for this exercise are available here:

https://keshvadi.github.io/security/labs/garage.tar

Unzip the downloaded file on your machine. This extracts three directories and two files:

- `bins_for_host`: Binaries to run on the server machine.
- `bins_for_client`: Binaries to run on the client (observer/attacker) machine.
- `pcaps`: Holds network traffic pcap files.
- `Dockerfile` and `docker-compose.yml`: Configuration file for the Docker environment.

2. Docker Setup

We use Docker to ensure a consistent environment. Make sure you set up this environment before beginning. Open a terminal in the folder where you unzipped the files and run the following command to build the project:

```
docker compose up -d --build
```

This will download necessary resources and start the server and client containers in the background.

1. Open two terminals.

2. Terminal 1 (Server): Run the following command to start the server container.

```
docker compose run --rm server bash
```

You should see a prompt like: `root@garage-server:/app#`

3. Terminal 2 (Client): Run the following command to start the client container.

```
docker compose run --rm client bash
```

You should see a prompt like: `root@garage-client:/app#`

You will run the `garage_server` in Terminal 1 and `garage_observer` (or `garage_attacker`) in Terminal 2. This setup allows you to correctly capture network traffic between the two containers.

Important Note: To facilitate automated grading, both the `garage_observer` and `garage_attacker` are programmed to automatically send a single character immediately after connecting (`'o'` for Observer and `'a'` for Attacker). You will see this initial 1-byte plaintext message at the start of your Wireshark captures. This is purely for marking purposes and can be ignored during your cryptographic analysis.

Assignment Versions

Version 1 (4 points)

The first version of the protocol does not use encryption. Your job is to figure out the basic protocol (how to connect, open doors, etc.) and then issue your own command by creating your own client.

Tip: Use this version to get comfortable with Wireshark, timestamps, and the basic command structure.

Step-by-Step Walkthrough:

In this first exercise, we will walk you through the entire process of running the experiment, capturing traffic, and launching an attack.

1. **Start the Server:**

In your **Server Terminal** (Terminal 1), run the server with the last 4 digits of your UCID (e.g., `5678`):

```
./garage_server_v1 5678
```

Note the port number the server prints out (e.g., "Server running on port 12345").

2. **Start Capturing Network Traffic:**

In your **Client Terminal** (Terminal 2), start `tcpdump` to record network traffic to a file named `v1.pcap`. We run this in the background (`&`) so we can keep using the terminal:

```
tcpdump -w v1.pcap &
```

3. **Run the Observer:**

Still in the **Client Terminal**, run the observer. You need the Server's IP (which is simply `garage-server` in our Docker setup) and your UCID:

```
./garage_observer_v1 garage-server 5678
```

Watch the output. You will see messages like "Opening Door" or "Turning on Lights" with timestamps.

4. **Stop the Capture:**

Once the observer finishes, bring `tcpdump` to the foreground and stop it (or just kill the process):

```
pkill tcpdump
```

You now have a file named `v1.pcap` in your current directory.

5. **Analyze the Traffic:**

Open `v1.pcap` in **Wireshark** on your host machine.

- **Filter the traffic:** Type `tcp.port == <PORT>` (use the port from Step 1) in the filter bar.
- **Follow the Stream:** Right-click one of the packets -> **Follow** -> **TCP Stream**.
- **Read the Protocol:** You will see the plain text commands sent by the observer (e.g., `CONNECT`, `OPEN`, etc.).
- **Extract the Hex:** Identify the specific bytes used to "Open Door". Select the text in the "Follow TCP Stream" window, switch the view to "Hex Dump" or "C Arrays" if needed, or simply look at the packet bytes in the main window. Right-click the payload bytes in the bottom pane -> **Copy** -> **...as a Hex Stream**.

6. **Launch the Attack:**

Now you know what bytes the server expects. Use the `garage_attacker` to send them.

- Identify the hex string for the "Open Door" command.
- Run the attacker in your **Client Terminal**:

  ```
  ./garage_attacker server <PORT>
  ```

- The program will wait for input. Paste the hex string for the `CONNECT` command and press Enter.

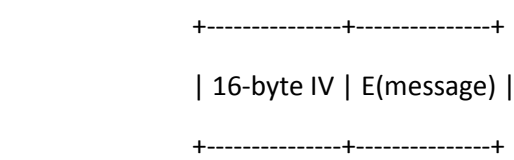
- Paste the hex string for the `OPEN DOOR` command and press Enter.
- If successful, the server will reply with: `SECURITY BREACH: DOOR OPEN! code: XXXXXX`.

Version 2 (8 points)

Version two is improved by using encryption! Unfortunately, the developers used **AES in ECB mode** (oops). There is a shared key between the garage door opener and the garage, and all messages are encrypted with 128-bit AES in ECB mode with that key. You do not need to figure out the key in order to figure out how to leave the door open.

Version 3 (8 points)

Version three is improved by using **128-bit AES in CBC mode** (much better). Better still, they use random per-message IVs. Each message sent from the client to the server has the following structure:



A message like "open" will now be sent as a 16-byte IV followed by the encryption of the message using the shared key and the sent IV. Again, you do not need to learn the key to attack the system.

Version 4 (8 points)

Version four is the same as version three, except that AES is used in **Counter Mode (CTR)**. This means it functions like a stream cipher, not a block cipher. The IV is still prefixed to the encrypted message. However, the observer program no longer reports what is happening when the garage door is being used. You will see commands in the packet capture, but you won't be able to relate packets to garage events.

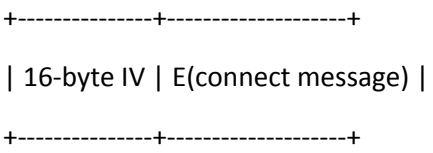
Think: What information gets leaked by a stream cipher as compared to a block cipher?

Version 5 (16 points)

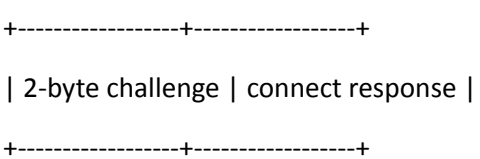
Version five is the same as version four, except that a **two-byte random challenge** is provided by the server to the client as a reply to the connect command. All subsequent commands must include this challenge as a prefix to the command, and it is encrypted as well.

Protocol Structure:

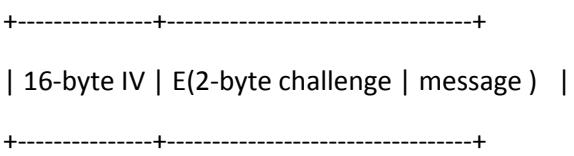
1. **Client Connects:**



2. **Server Response:**



3. **Client Sends Command:**



Each connection results in a new random challenge. If you run the observer, the challenge you capture won't be the same as the one you receive when you try to replay the message. Nevertheless, the fact that it is a **stream cipher** allows for a **malleability-based attack**.

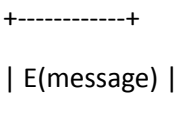
> *Hint: You may find it useful to copy hex commands from Wireshark to a file and add spaces, so you know you are modifying the correct components.*

Version 6 (16 points)

Version six is a modification of version four, except that instead of using a random IV, there is a **fixed IV of all zeros**. Every message is encrypted with the same key and IV using AES-CTR.

The catch is that during the observer phase, you **won't see any garage door opening commands**. You can keep running it, but an "open" command will never happen. You must figure out how to craft such a message yourself.

Message Structure (IV is not transmitted):



> *Hint: The use of counter mode with IV reuse results in what problem?*

Good Luck!